2

PART 1: FOUNDATION

Why purpose comes before design

Why does skipping the vision step cost more than writing the vision would have?

The Riverside Clinics team has learned the lesson from the last chapter. Their next request to the AI assistant names actors, data, and a rule, not just a feature label. A product manager reads it over and likes the improvement. Then a colleague asks a question the request still doesn't answer: why are we building this, and for whom, exactly?

The product manager opens a blank document and writes a sentence in under five minutes. "Build a system that lets patients book appointments online instead of calling the clinic." It reads like an answer. It is the Chapter 1 request with a clause bolted on.

CHAPTER PROMISE

By the end of this chapter, you can write a vision statement that names the actual problem behind a feature request instead of the feature itself, and use it to surface a barrier your original request never mentioned, before a single line of code gets written around the wrong one.

Reading time: 11 minutes

A sentence that restates the request instead of explaining it

The product manager's sentence looks like a vision. It reads in prose instead of a ticket title, and it names patients and calling and booking online. But it answers a question nobody asked. It says what the system will do. It says nothing about why patients call today, or whether every patient who calls has the same reason for it. A team could write ten versions of that sentence, each better worded than the last, and learn nothing new about the people they are building for.

That gap costs more once it's found late than it would have cost to close early. Nobody asked why patients pick up the phone at all, rather than walking up to the front desk. So nobody could know there might be more than one reason, or that one of those reasons has nothing to do with convenience. By the time the booking flow is designed and built, finding a missed reason means reopening a decision that now has a database schema and a user interface built around it, not adding a paragraph to a document.

Scope decided late is scope decided by default. If nobody writes down what the system is and isn't responsible for, that boundary still gets set, just later, and by whatever the first architecture happens to support rather than by anyone's judgment. A vision written before any of that exists turns the boundary into a decision the team makes on purpose.

What a vision actually answers

A **vision** is a short, explicit statement of why a system exists and who it exists for: the problem it solves, the people who have that problem, and the boundary around what the system is responsible for and what it isn't. What the system does is a separate question, answered by the layers built on top of it, starting with the next chapter.

Vision and requirements get confused constantly, because both eventually sit in the same folder. They aren't interchangeable. A requirement is testable: the system must accept a booking request up to ninety days in advance. A vision explains why that requirement is worth having at all: patients are giving up on booking, and the clinic wants to know why before it decides how far out a slot should be offered. Requirements change often, as the team learns more about the solution it's building. Vision changes rarely, because the underlying problem rarely does.

> **A vision answers why. Everything written after it, requirements, use cases, a domain model, answers what and how. Get why wrong, and precision about what and how won't save the project.**

One question, asked at several sizes

Vision doesn't only apply to a single feature. Riverside Clinics, the organization, has a reason it exists: patients need care, and the clinics providing it need to run without every hour going to logistics instead of medicine. The patient portal, one product inside that organization, has a narrower reason: the phone is the slowest and least private way for a patient to reach the clinic, and the portal is meant to be the alternative. Online appointment booking, one feature inside that portal, has a narrower reason still, the one this chapter is building toward.

Each level answers the same question, why does this exist, at its own scope. The feature's vision has to fit inside the product's vision, and the product's vision has to fit inside the organization's. A booking feature that solves a problem the patient portal was never meant to solve is a sign that someone skipped a level, not that the feature is wrong on its own terms.

Finding the reason nobody wrote down

A vision that only restates the request usually comes from writing it alone, at a desk, from the ticket. The fix is a short conversation with the people closest to the current, broken way of doing things, run before the vision gets written rather than after.

1. Gather the people who deal with the current process daily, not only whoever requested the feature: front desk staff, a nurse or clinician, and a patient if one is reachable.
2. Ask three questions, in this order: what's wrong with how this works today, who feels that most directly, and why hasn't it already been fixed.
3. Listen for an answer that changes or adds to the original request, not one that only confirms it. A room that agrees with the ticket in the first two minutes hasn't been asked the right question yet.
4. Write the answer down as a sentence naming the specific problem and the specific people affected, not the feature that might solve it.

One short meeting with the right people in the room is usually enough. The cost is a meeting. The alternative is finding out after launch.

The Riverside booking vision, badly and then well

The product manager's sentence from the start of this chapter is the poorly written version below. Run through the conversation from the previous page, it doesn't survive intact.

The front desk lead is in the room, and so is a nurse who spends part of every shift on the phone. The nurse says the quiet part out loud: patients calling to book aren't only losing time on hold. Some patients never call, and never walk up to the counter either, because doing either one means stating the reason for the visit where staff, and sometimes other patients in the waiting room, can hear it, a mental health follow-up, or a test result they'd rather not announce out loud. Those patients don't complain about wait times. They just don't book, and the clinic never learns they needed care. That one sentence turns a vague convenience problem into a specific one.

Poor vision

```
"Build a system that lets patients book appointments online instead of calling the clinic."
```

Working vision

```
Patients currently book by phone or in person at the front desk. Which doctor is at which location on which day is knowledge only the front desk holds, so a patient booked at the wrong site arrives and is turned away. Both routes also require stating the reason for a visit where staff, and sometimes other patients, can hear it. Some patients tolerate that. Others avoid booking entirely rather than disclose it aloud, and the clinic never learns they needed care.

We want patients to book, reschedule, and cancel appointments online, privately, without narrating the reason for the visit to anyone. This is for patients who currently go without care rather than disclose it in person, and for front desk staff spending a large share of their day on calls a private alternative could remove.

This system is responsible for booking, availability, and confirmation. It is not responsible for billing, medical records, or clinical intake, which stay with the systems that already handle them. Success looks like fewer routine calls to the front desk, and bookings for visit types patients previously avoided scheduling in person.
```

Where a vision statement stops helping

▲ **WARNING**

A vision that names a problem is not proof the problem was diagnosed correctly. The stakeholder conversation in this chapter surfaces a barrier the original request missed; it doesn't confirm that the barrier it surfaces is the only one, or the biggest one. Writing an assumption down makes it visible. It doesn't make it true.

Treating vision as something written once and never revisited	Riverside's patient population, referral mix, and locations change over a few years; a vision frozen from the launch conversation keeps the team solving a version of the problem that no longer exists.
Writing vision by committee, aiming for agreement rather than clarity	A vision drafted to satisfy everyone in the room tends to soften back into the restated request this chapter opened with, because specificity is usually what disagreement gets negotiated away.

Field guide: the vision-document outline

- ☐ **The Problem.** What's unsatisfactory about how this works today, stated as a fact about the world, not a missing feature?
- ☐ **The Stakeholders.** Who has this problem directly, and who else is affected, including anyone who currently avoids the process rather than tolerates it?
- ☐ **The Scope.** What is this system responsible for, and what is explicitly left to something else?
- ☐ **The Approach.** In one or two sentences, what general approach addresses the problem, without describing the implementation?
- ☐ **The Success Criteria.** What observable change tells the team the problem is actually being solved?
- ☐ **Constraints.** What has already been decided, or must remain true, regardless of how the rest gets built?

Fill in one sentence per line before writing anything longer. A line you can't fill in is a stakeholder conversation you haven't had yet.

What to remember

- A vision answers why a system exists and for whom. What the system does belongs to the layers built on top of it.
- A vision that only restates the feature request has the shape of an answer without any of the content; it teaches the team nothing it didn't already know.
- The barrier a request never mentions usually surfaces in conversation with the people closest to the current process, not at a desk with the ticket alone.
- Vision fixes scope before architecture does it by default. Skipping it hands that decision to whatever gets built first.
- The same question, why does this exist, applies at the scale of a feature, a product, and an organization, each nested inside the one above it.

QUESTIONS FOR YOUR TEAM

- If you asked three people on your team why your current project exists, would they give the same answer, or three different ones?
- Is there a group of users your last feature request never mentioned, because nobody in the room had asked them anything?

ONE ACTION TO TAKE NEXT

Take the vision statement for whatever you're building now, or write one in five minutes if none exists. Read it against the field guide on the previous page. If it only restates the feature, find one person closest to the current process and ask the three questions from this chapter before you write the next draft.

TRANSITION. A vision names the problem and the people who have it. It doesn't yet say what the system must do about them, precisely enough for an AI assistant to build it correctly. The next chapter turns that vision into requirements.

Why purpose comes before design ends here.